GURU GHASIDAS VISHWAVIDYALAYA

KONI,BILASPUR (C.G.)

DEPARTMENT OF CSIT

SESSION – 2025-2026

PRACTICAL ASSIGNMENT

NAME : KOYANA RATHORE

COURSE : B.Sc. (H) C.S. (4th SEMESTER)

ENROL. NO. : GGV/24/05133

ROLL NO. : 24070133

SUBJECT : OOPs IN JAVA

GUIDED BY:

DR. RATNESH SHRIVASTAVA

1. **Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.**

Code:

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

public class FarmersDataHandler {

    static class Farmer {
        String name;
        String fruit;
        String city;
        int quantity;

        Farmer(String name, String fruit, String city, int quantity) {
            this.name = name;
            this.fruit = fruit;
            this.city = city;
            this.quantity = quantity;
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();

        // Input data for 10 farmers
        for (int i = 1; i <= 10; i++) {
            System.out.println("Enter details for Farmer " + i);
            System.out.print("Name: ");
            String name = scanner.nextLine();

            System.out.print("Fruit: ");
            String fruit = scanner.nextLine();

            System.out.print("City: ");
            String city = scanner.nextLine();
```

```java
            System.out.print("Quantity: ");
            int quantity = Integer.parseInt(scanner.nextLine());

            farmers.add(new Farmer(name, fruit, city, quantity));
        }

        String filename = "farmers_data.txt"; // Single filename for both data states

        // Save data before sorting
        writeToFile(farmers, filename, "Data Before Sorting");

        // Sort farmers based on quantity in descending order
        farmers.sort(Comparator.comparingInt((Farmer f) -> f.quantity).reversed());

        // Save data after sorting
        writeToFile(farmers, filename, "Data After Sorting");

        // Search functionality
        System.out.print("Enter farmer's name to search: ");
        String searchName = scanner.nextLine();

        boolean found = false;
        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("Farmer found: " + f.name + ", Fruit: " + f.fruit + ",
City: " + f.city + ", Quantity: " + f.quantity);
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Farmer with name " + searchName + " not found.");
        }

        scanner.close();
    }

    private static void writeToFile(List<Farmer> farmers, String filename, String header)
{
        try (FileWriter writer = new FileWriter(filename, true)) {
            writer.write(header + "\n");
            writer.write(String.format("%-15s %-15s %-15s %-10s\n", "Name", "Fruit",
"City", "Quantity"));
            for (Farmer f : farmers) {
                writer.write(String.format("%-15s %-15s %-15s %-10d\n", f.name, f.fruit,
f.city, f.quantity));
            }
            writer.write("\n");
        }
```

```java
catch (IOException e) {
            System.out.println("An error occurred while writing to file.");
        }
    }
}
```

Output:

```
PROBLEMS 1    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

AppData\Roaming\Code\User\workspaceStorage\e2585bc5dd2416ba1ea38c213937ee41\redhat.java\jdt_ws\hellow
orld_4f93cdcf\bin' 'FarmersDataHandler'
Enter details for Farmer 1
Name: customer 1
Fruit: mango
City: korba
Quantity: 13
Enter details for Farmer 2
Name: customer 2
Fruit: apple
City: bilaspur
Quantity: 23
Enter details for Farmer 3
Name: customer 3
Fruit: pineapple
City: raipur
Quantity: 34
Enter farmer's name to search: customer 2
Farmer found: customer 2, Fruit: apple, City: bilaspur, Quantity: 23
PS D:\books\java\helloworld>
```

List:

farmers_data_unique.txt

```
Data Before Sorting
Name            Fruit           City            Quantity
customer 1      mango           korba           13
customer 2      apple           bilaspur        23
customer 3      pineapple       raipur          34

Data After Sorting
Name            Fruit           City            Quantity
customer 3      pineapple       raipur          34
customer 2      apple           bilaspur        23
customer 1      mango           korba           13
```

2. **Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.**

Code :

```
import java.util.Scanner;

public class FruitManagementSystem {

    static class Fruit {
        String name;
        int quantity;
        String quality;

        Fruit(String name, int quantity, String quality) {
            this.name = name;
            this.quantity = quantity;
            this.quality = quality;
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input details for Mango
        Fruit mango = inputFruitDetails(scanner, "Mango");

        // Input details for Apple
        Fruit apple = inputFruitDetails(scanner, "Apple");

        // Input details for Papaya
        Fruit papaya = inputFruitDetails(scanner, "Papaya");

        // Array of fruits for searching
        Fruit[] fruits = {mango, apple, papaya};

        // Prompt user for search criteria
        System.out.print("Enter the name of the fruit to search (e.g., Mango, Apple,
Papaya): ");
        String searchFruit = scanner.nextLine();

        System.out.print("Enter the desired quality to search for (good, bad, average):
");
        String searchQuality = scanner.nextLine();

        boolean found = false;

        // Search for matching fruit and quality
        for (Fruit f : fruits) {
```

```java
            if (f.name.equalsIgnoreCase(searchFruit) &&
f.quality.equalsIgnoreCase(searchQuality)) {
                System.out.println("Match Found: " + f.name + " with quality '" +
f.quality + "' and quantity " + f.quantity);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("No matching fruit with the specified quality was
found.");
        }

        scanner.close();
    }

    private static Fruit inputFruitDetails(Scanner scanner, String fruitName) {
        System.out.println("Please provide details for " + fruitName + ":");
        System.out.print("Enter quantity (if greater than 1000, it will be stored as
half): ");
        int quantity = Integer.parseInt(scanner.nextLine());

        // Adjust quantity if greater than 1000
        if (quantity > 1000) {
            quantity = quantity / 2;
        }

        System.out.print("Enter quality (good, bad, average): ");
        String quality = scanner.nextLine();

        return new Fruit(fruitName, quantity, quality);
    }
}
```

Output :

```
PROBLEMS 1    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

System'
Please provide details for Mango:
Enter quantity (if greater than 1000, it will be stored as half): 1370
Enter quality (good, bad, average): average
Please provide details for Apple:
Enter quantity (if greater than 1000, it will be stored as half): 600
Enter quality (good, bad, average): good
Please provide details for Papaya:
Enter quantity (if greater than 1000, it will be stored as half): 450
Enter quality (good, bad, average): bad
Enter the name of the fruit to search (e.g., Mango, Apple, Papaya): mango
Enter the desired quality to search for (good, bad, average): good
No matching fruit with the specified quality was found.
```

3. **There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively. The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.**

Code:

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class OrchardTreeView {

    static class Orchard {
        String name;
        String fruit;
        String quality;

        Orchard(String name, String fruit, String quality) {
            this.name = name;
            this.fruit = fruit;
            this.quality = quality;
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Initialize the orchards as per the given data
        List<Orchard> orchards = new ArrayList<>();
        orchards.add(new Orchard("A", "Apple", "good"));
        orchards.add(new Orchard("B", "Litchi", "good"));
        orchards.add(new Orchard("C", "Mango", "good"));
        orchards.add(new Orchard("D", "Apple", "bad"));

        // Display the orchards in a tree view
        System.out.println("Orchard Tree View:");
        System.out.println("Orchards:");
        for (Orchard orchard : orchards) {
            System.out.println("├── Orchard " + orchard.name);
            System.out.println("│   ├── Fruit: " + orchard.fruit);
            System.out.println("│   └── Quality: " + orchard.quality);
        }

        // Input from user for searching a fruit
        System.out.print("\nEnter the fruit name to search for: ");
        String searchFruit = scanner.nextLine();
```

```java
 // Search for the fruit in the orchards
        boolean found = false;
        System.out.println("\nSearch Results:");
        for (Orchard orchard : orchards) {
            if (orchard.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println("Orchard " + orchard.name + " contains " +
orchard.fruit + " with quality '" + orchard.quality + "'");
                found = true;
            }
        }

        if (!found) {
            System.out.println("No orchards contain the fruit: " + searchFruit);
        }

        scanner.close();
    }
}
```

Output :

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS          Run: OrchardTreeView

PS D:\books\java\helloworld>  & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotspot\bin\java.exe' '-XX:+Shc
etailsInExceptionMessages' '-cp' 'C:\Users\Admin\AppData\Roaming\Code\User\workspaceStorage\e2585bc5dd2416ba1ea3
37ee41\redhat.java\jdt_ws\helloworld_4f93cdcf\bin' 'OrchardTreeView'
Orchard Tree View:
Orchards:
├── Orchard A
│   ├── Fruit: Apple
│   └── Quality: good
├── Orchard B
│   ├── Fruit: Litchi
│   └── Quality: good
├── Orchard C
│   ├── Fruit: Mango
│   └── Quality: good
├── Orchard D
│   ├── Fruit: Apple
│   └── Quality: bad

Enter the fruit name to search for: apple

Search Results:
Orchard A contains Apple with quality 'good'
Orchard D contains Apple with quality 'bad'
PS D:\books\java\helloworld>
```

4. Take a map provided as a graph & display all the subtrees of connected points.

Code :

```
import java.util.*;

public class SubtreeGraph {

    private int vertices;
    private ArrayList<ArrayList<Integer>> adj;

    // Constructor
    SubtreeGraph(int v) {
        vertices = v;
        adj = new ArrayList<>();

        for (int i = 0; i < v; i++) {
            adj.add(new ArrayList<>());
        }
    }

    // Add edge
    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }

    // Find subtrees using iterative DFS
    void findSubtrees() {

        boolean visited[] = new boolean[vertices];
        int subtreeCount = 1;

        for (int i = 0; i < vertices; i++) {

            if (!visited[i]) {

                Stack<Integer> stack = new Stack<>();
                List<Integer> nodes = new ArrayList<>();
                List<String> edges = new ArrayList<>();

                stack.push(i);
                visited[i] = true;

                while (!stack.isEmpty()) {

                    int current = stack.pop();
                    nodes.add(current);

                    for (int neighbor : adj.get(current)) {
```

```java
                        if (!visited[neighbor]) {
                            visited[neighbor] = true;
                            stack.push(neighbor);

                            // storing spanning tree edge
                            edges.add(current + " - " + neighbor);
                        }
                    }
                }

                System.out.println("\nSubtree " + subtreeCount);

                System.out.println("Nodes:");
                for (int n : nodes) {
                    System.out.print(n + " ");
                }

                System.out.println("\nEdges:");
                for (String e : edges) {
                    System.out.println(e);
                }

                subtreeCount++;
            }
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int v = sc.nextInt();

        SubtreeGraph g = new SubtreeGraph(v);

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        System.out.println("Enter edges (u v):");

        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v2 = sc.nextInt();
            g.addEdge(u, v2);
        }

        System.out.println("\nSubtrees of Connected Points:");
        g.findSubtrees();
    }
}
```

Ouput :

```
AppData\Roaming\Code\User\workspaceStorage\e2585bc5dd2416ba1ea38c213937ee41\redhat.java\jdt_ws\hellow
orld_4f93cdcf\bin' 'SubtreeGraph'
Enter number of vertices: 7
Enter number of edges: 5
Enter edges (u v):
0 1
1 2
3 4
5 6
2 0

Subtrees of Connected Points:

Subtree 1
Nodes:
0 2 1
Edges:
0 - 1
0 - 2

Subtree 2
Nodes:
3 4
Edges:
3 - 4

Subtree 3
Nodes:
5 6
Edges:
5 - 6
PS D:\books\java\helloworld>
```

5. **Enter entry point & goal point & produce the matching subtree of the graph.**

Code:

```
import java.util.*;

class Node {
    int id;
    String value;
    List<Integer> edges;

    public Node(int id, String value) {
        this.id = id;
        this.value = value;
        this.edges = new ArrayList<>();
    }
}

class Graph {
    private Map<Integer, Node> nodes;

    public Graph() {
        this.nodes = new HashMap<>();
    }

    public void addNode(int id, String value) {
        nodes.put(id, new Node(id, value));
    }

    public void addEdge(int id1, int id2) {
        nodes.get(id1).edges.add(id2);
        nodes.get(id2).edges.add(id1);
    }

    public List<Integer> findPath(int startId, int goalId) {
        Map<Integer, Integer> parentMap = new HashMap<>();
        Queue<Integer> queue = new LinkedList<>();
        Set<Integer> visited = new HashSet<>();

        queue.offer(startId);
        visited.add(startId);
        parentMap.put(startId, null);

        while (!queue.isEmpty()) {
            int current = queue.poll();
            if (current == goalId) {
                // Build path
                List<Integer> path = new ArrayList<>();
                for (Integer at = goalId; at != null; at = parentMap.get(at)) {
                    path.add(at);
```

```
                }
                Collections.reverse(path);
                return path;
            }
            for (int neighbor : nodes.get(current).edges) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    parentMap.put(neighbor, current);
                    queue.offer(neighbor);
                }
            }
        }
        // No path found
        return Collections.emptyList();
    }

    public Map<Integer, Node> getNodes() {
        return nodes;
    }
}

public class GraphMatchingSubtree {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Graph graph = new Graph();

        // Sample graph setup
        graph.addNode(1, "A");
        graph.addNode(2, "B");
        graph.addNode(3, "C");
        graph.addNode(4, "D");
        graph.addNode(5, "E");

        graph.addEdge(1, 2);
        graph.addEdge(1, 3);
        graph.addEdge(2, 4);
        graph.addEdge(3, 4);
        graph.addEdge(4, 5);

        System.out.println("Available nodes:");
        for (Map.Entry<Integer, Node> entry : graph.getNodes().entrySet()) {
            System.out.println("ID: " + entry.getKey() + ", Value: " +
entry.getValue().value);
        }

        System.out.print("Enter entry point node ID: ");
        int startId = scanner.nextInt();

        System.out.print("Enter goal point node ID: ");
        int goalId = scanner.nextInt();

        Map<Integer, Node> nodes = graph.getNodes();
```

```java
        // Check if nodes exist
        if (!nodes.containsKey(startId) || !nodes.containsKey(goalId)) {
            System.out.println("Invalid node IDs entered.");
            scanner.close();
            return;
        }

        // Find path between start and goal
        List<Integer> path = graph.findPath(startId, goalId);

        if (path.isEmpty()) {
            System.out.println("No path exists between the given nodes.");
        } else {
            System.out.println("Path between nodes: " + path);
        }
        scanner.close();
    }
}
```

Output:

```
\jdk-17.0.18.8-hotspot\bin\java.exe  -XX:+ShowCodeDetailsInExceptionMessages  -cp  C:\Users\Admin\
AppData\Roaming\Code\User\workspaceStorage\e2585bc5dd2416ba1ea38c213937ee41\redhat.java\jdt_ws\hellow
orld_4f93cdcf\bin' 'GraphMatchingSubtree'
Available nodes:
ID: 1, Value: A
ID: 2, Value: B
ID: 3, Value: C
ID: 4, Value: D
ID: 5, Value: E
Enter entry point node ID: 3
Enter goal point node ID: 1
Path between nodes: [3, 1]
PS D:\books\java\helloworld>
```

6. **Give the weightage for each edge of nodes, and produce the shortest path from start to end node.**

Code:

```
import java.util.*;

public class ShortestPath {
    static class Edge {
        int target;
        int weight;

        Edge(int target, int weight) {
            this.target = target;
            this.weight = weight;
        }
    }

    static class Graph {
        int vertices;
        LinkedList<Edge>[] adjList;

        @SuppressWarnings("unchecked")
        Graph(int vertices) {
            this.vertices = vertices;
            adjList = new LinkedList[vertices];
            for (int i = 0; i < vertices; i++) {
                adjList[i] = new LinkedList<>();
            }
        }

        void addEdge(int src, int dest, int weight) {
            adjList[src].add(new Edge(dest, weight));
            // For undirected graph, uncomment below:
            // adjList[dest].add(new Edge(src, weight));
        }

        int[] shortestPath(int start, int end) {
            int[] dist = new int[vertices];
            int[] prev = new int[vertices];
            boolean[] visited = new boolean[vertices];

            PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a ->
a[1]));
            Arrays.fill(dist, Integer.MAX_VALUE);
            Arrays.fill(prev, -1);

            dist[start] = 0;
            pq.offer(new int[] { start, 0 });
```

```java
            while (!pq.isEmpty()) {
                int[] current = pq.poll();
                int u = current[0];

                if (visited[u]) continue;
                visited[u] = true;

                for (Edge edge : adjList[u]) {
                    int v = edge.target;
                    int weight = edge.weight;
                    if (!visited[v] && dist[u] + weight < dist[v]) {
                        dist[v] = dist[u] + weight;
                        prev[v] = u;
                        pq.offer(new int[] { v, dist[v] });
                    }
                }
            }

            // Reconstruct path
            List<Integer> path = new ArrayList<>();
            for (int at = end; at != -1; at = prev[at]) {
                path.add(at);
            }
            Collections.reverse(path);

            if (dist[end] == Integer.MAX_VALUE) {
                System.out.println("No path exists from " + start + " to " + end);
                return new int[0];
            } else {
                System.out.println("Shortest distance from " + start + " to " + end + "
is " + dist[end]);
                System.out.println("Path: " + path);
                return path.stream().mapToInt(i -> i).toArray();
            }
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int numVertices = scanner.nextInt();

        Graph g = new Graph(numVertices);

        System.out.println("Enter number of edges: ");
        int numEdges = scanner.nextInt();

        System.out.println("Enter edges in format: source destination weight");
        for (int i = 0; i < numEdges; i++) {
            int src = scanner.nextInt();
            int dest = scanner.nextInt();
```

```java
            int weight = scanner.nextInt();
            g.addEdge(src, dest, weight);
        }

        System.out.print("Enter start node: ");
        int startNode = scanner.nextInt();

        System.out.print("Enter end node: ");
        int endNode = scanner.nextInt();

        g.shortestPath(startNode, endNode);

        scanner.close();
    }
}
```

Output:

```
Enter edges in format: source destination weight
0 1 4
0 2 4
1 2 2
1 3 5
2 3 8
2 4 10
3 4 2
3 5 6
4 5 2
Enter start node: 3
Enter end node: 5
Shortest distance from 3 to 5 is 4
Path: [3, 4, 5]
PS D:\books\java\helloworld>
```

7. **Write a program to demonstrate the use of all keywords made for exception handling.**

Code:

```
public class ExceptionHandlingKeywordsDemo {

    // Method that declares it may throw an exception
    public static void riskyMethod() throws ArithmeticException {
        int a = 10, b = 0;
        // Using throw keyword to throw an exception explicitly
        if (b == 0) {
            throw new ArithmeticException("Division by zero");
        }
        int result = a / b;
        System.out.println("Result: " + result);
    }

    public static void main(String[] args) {
        try {
            // Using try keyword to catch exceptions
            riskyMethod();
        } catch (ArithmeticException e) {
            // Using catch keyword to handle exception
            System.out.println("Caught an exception: " + e.getMessage());
        } finally {
            // Using finally keyword - code that always executes
            System.out.println("Finally block executed");
        }

        // Demonstrating throws keyword by calling a method that throws an exception
        try {
            methodThatThrows();
        } catch (Exception e) {
            System.out.println("Caught exception from methodThatThrows: " +
e.getMessage());
        }
    }

    // Method that uses throws keyword
    public static void methodThatThrows() throws Exception {
        // Using throw keyword to throw an exception
        throw new Exception("This is an exception thrown from methodThatThrows");
    }
}
```

Output:

```
ngKeywordsDemo'
Caught an exception: Division by zero
Finally block executed
Caught exception from methodThatThrows: This is an exception thrown from methodThatThrows
PS D:\books\java\helloworld>
```

8. **Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.**

Code:

```
import java.util.Scanner;

// Custom exception class
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}

public class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    public void withdraw(double amount) throws NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException("Insufficient balance. Cannot withdraw " +
amount);
        }
        balance -= amount;
        System.out.println("Withdrawal successful. Remaining balance: " + balance);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        BankAccount account = new BankAccount(1000.0); // Initial balance

        try {
            System.out.print("Enter amount to withdraw: ");
            double amount = scanner.nextDouble();

            account.withdraw(amount);
        } catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Invalid input. Please enter a numeric value.");
        } finally {
            scanner.close();
        }
    }
}
```

Output:

```
AppData\Roaming\Code\User\workspaceStorage\e2585bc5dd2416ba1ea38c213937ee41\redhat.java\jdt_ws\hellow
orld_4f93cdcf\bin' 'BankAccount'
Enter amount to withdraw: 270
Withdrawal successful. Remaining balance: 730.0
PS D:\books\java\helloworld>
```

9. Write a program which enables to use a platform by a train one at a time.

Code:

```
class Platform {
    public synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is using the platform.");
        try {
            Thread.sleep(2000); // Simulate the train using the platform
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println(trainName + " has left the platform.");
    }
}

class Train extends Thread {
    private Platform platform;
    private String trainName;

    public Train(Platform platform, String trainName) {
        this.platform = platform;
        this.trainName = trainName;
    }

    @Override
    public void run() {
        platform.usePlatform(trainName);
    }
}

public class PlatformUsage {
    public static void main(String[] args) {
        Platform platform = new Platform();

        // Creating multiple train threads
        Train train1 = new Train(platform, "Train 1");
        Train train2 = new Train(platform, "Train 2");
        Train train3 = new Train(platform, "Train 3");

        // Starting the trains
        train1.start();
        train2.start();
        train3.start();
    }
}
```

Output:

```
Train 1 is using the platform.
Train 1 has left the platform.
Train 2 is using the platform.
Train 2 has left the platform.
Train 3 is using the platform.
Train 3 has left the platform.
PS D:\books\java\helloworld>
```

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and chef deliver orders if he is free.

Code:

```
import java.util.Scanner;

class Kitchen {
    private boolean isBusy = false;

    public synchronized boolean acceptOrder(String order) {
        if (isBusy) {
            return false; // Kitchen is busy
        } else {
            isBusy = true;
            System.out.println("Order accepted: " + order);
            return true;
        }
    }

    public synchronized void deliverOrder(String order) {
        System.out.println("Chef is delivering order: " + order);
        try {
            Thread.sleep(2000); // Simulate time taken to deliver
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println("Order delivered: " + order);
        isBusy = false;
    }
}

public class Restaurant {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Kitchen kitchen = new Kitchen();

        System.out.print("Enter number of orders to process: ");
        int numberOfOrders = scanner.nextInt();
        scanner.nextLine(); // Consume newline

        for (int i = 0; i < numberOfOrders; i++) {
            System.out.print("Enter order " + (i + 1) + ": ");
            String order = scanner.nextLine();

            // Waiter accepts order if kitchen is free
            if (kitchen.acceptOrder(order)) {
                // Chef delivers order in a separate thread
                Thread chefThread = new Thread(() -> {
                    kitchen.deliverOrder(order);
                });
```

```java
                chefThread.start();

                try {
                    chefThread.join(); // Wait for the delivery to complete before
accepting next order
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            } else {
                System.out.println("Kitchen is busy. Cannot accept the order at this
moment.");
                i--; // Retry the same order
            }
        }

        scanner.close();
    }
}
```

Output:

```
Storage\e2585bc5dd2416ba1ea38c213937ee41\redhat.java\jdt_ws\helloworld_4f93cdcf\bin' 'Resta
Enter number of orders to process: 3
Enter order 1: pizza
Order accepted: pizza
Chef is delivering order: pizza
Order delivered: pizza
Enter order 2: pasta
Order accepted: pasta
Chef is delivering order: pasta
Order delivered: pasta
Enter order 3: carbonara
Order accepted: carbonara
Chef is delivering order: carbonara
Order delivered: carbonara
PS D:\books\java\helloworld>
```